Operating system principles 2

Paul Blondel

CNAM

September 1st, 2018

'(Kenneth) Thompson and (Dennis) Ritchie were among the first to realize that hardware and compiler technology had become good enough that an entire operating system could be written in C, and by 1978 the whole environment had been successfully ported to several machines of different types.' Eric S. Raymond

Recall about the *compiler* and the *link editor*:

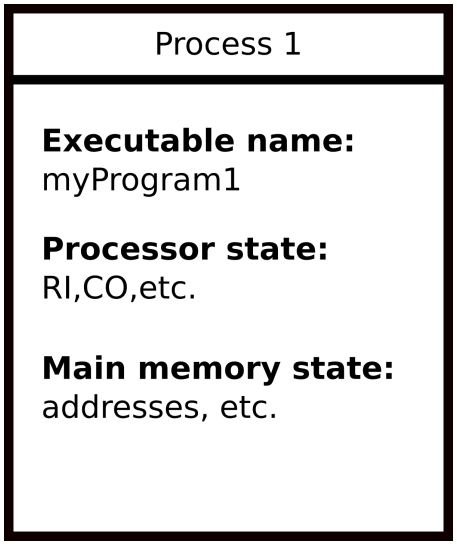
- The executable is produced by the *compiler* and the *link editor*
- The *loader* loads it into the *main memory*
- When the executable start being *executed*:
  1. The first instruction is set in the *RI* register
  2. The address of the next instruction is set in the *CO* register
  3. When executed: new instruction is set in *RI*
  4. It's new next instruction address is set in *CO*
  5. etc

So what is a process?

A process is an instance of an executable, it represents:

- the state of the executable **in the processor unit**
- the state of the executable **in the main memory**

Let's represent a *process* as follows:



A process can be in one of these three different **states**:

1. **elected**: the process is **executed by the processor**
2. **blocked**: the process is **waiting for a resource or another process**
3. **ready**: the process **can be elected**, but the processor is busy

A process can be in one of these three different **states**:

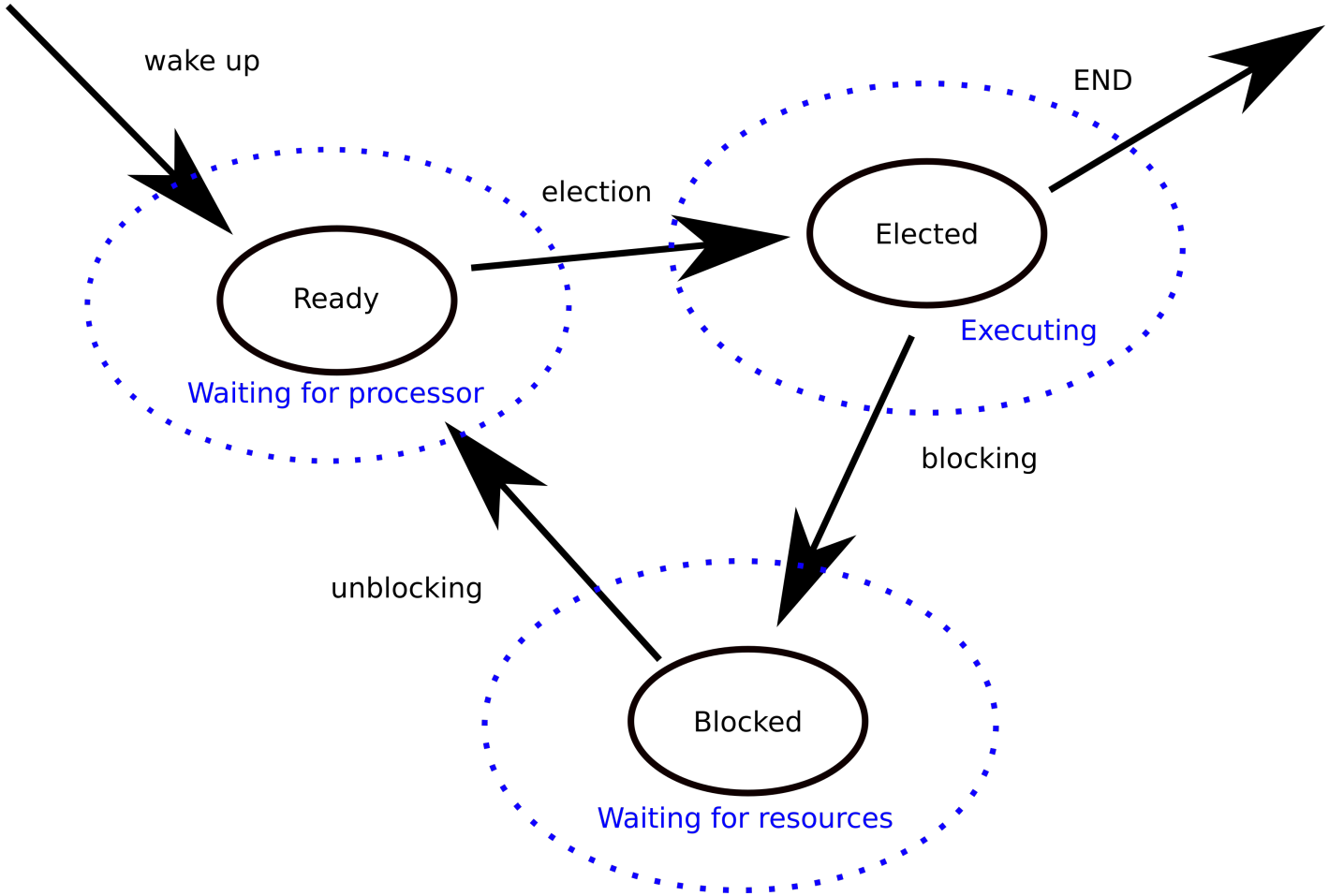
1. **elected**: the process is **executed by the processor**
2. **blocked**: the process is **waiting for a resource or another process**
3. **ready**: the process **can be elected**, but the processor is busy

When a process goes...

- ...from *ready state* to *elected state*: **election operation**
- ...from *elected state* to *blocked state*: **blocking operation**
- ...from *blocked state* to *ready state*: **unblocking operation**

Processes

Are always created in *ready* state and always finishes in *elected* state[1]



[1] Except in case of anomalies

When *launching* an executable:

1. The *loader* **loads in *main memory* the executable**
2. And **create a *Process Control Block*** (or ***PCB***)

When *launching* an executable:

1. The *loader* **loads in *main memory* the executable**
2. And **create a *Process Control Block* (or *PCB*)**

A *PCB* contains the following information:

- a **unique process identifier**
- a **processor context or state** (value of CO/RI, etc)
- a **memory context or state**:
  - where to get the executable instructions
  - where to get the associated data
- **statistics**
- **scheduling information**
- **resource information**

A *process* can create one or more processes[2]:

- The creating process is called the **father process**
- Created processes are called **child processes**

Depending on the system, a child *process*:

- may **inherit from source data and context** of the creator
- can execute a **totally different program** (Dec Vms system)
- can execute a **clone** (same data and code) (UNIX systems)

Note

In some systems the father waits for the children to end to resume

[2] Child processes can also create processes and become fathers, etc.

The *destruction* of a *process* happens when:

- The *process* has **terminated its execution**
  - It calls the *exit()* function on UNIX
- The *process* **gets an unsolvable error**
- Another *process* asks the ***destruction* of the *process***
  - Example: the *kill* command on UNIX

The *destruction* of a *process* happens when:

- The *process* has **terminated its execution**
  - It calls the *exit()* function on UNIX
- The *process* **gets an unsolvable error**
- Another *process* asks the ***destruction* of the *process***
  - Example: the *kill* command on UNIX

What happens during the *destruction*?

- The ***main memory* context is destroyed** (release memory, etc)
- ***Resources* are freed** and can be used by other processes
- The ***PCB* is destroyed**

What is suspending a process?

It is **momentarily stop the execution of a process** to resume later

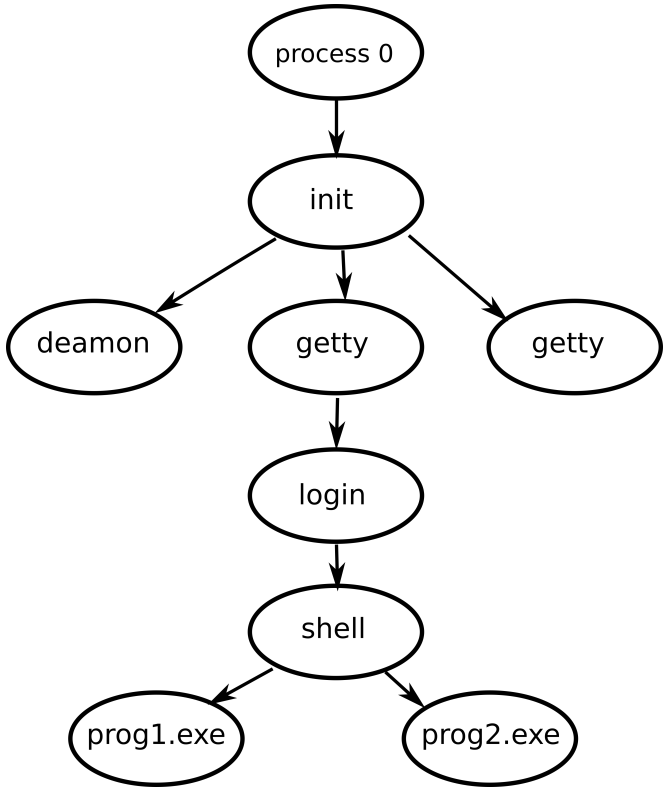
What happen during the suspension?

- *Processor* and *memory* contexts are stored in *PCB*
- The state of the *process* becomes *blocked*

Note

UNIX systems are entirely built around the notion of *processes*

When running a UNIX system there is the following *process* dependencies:



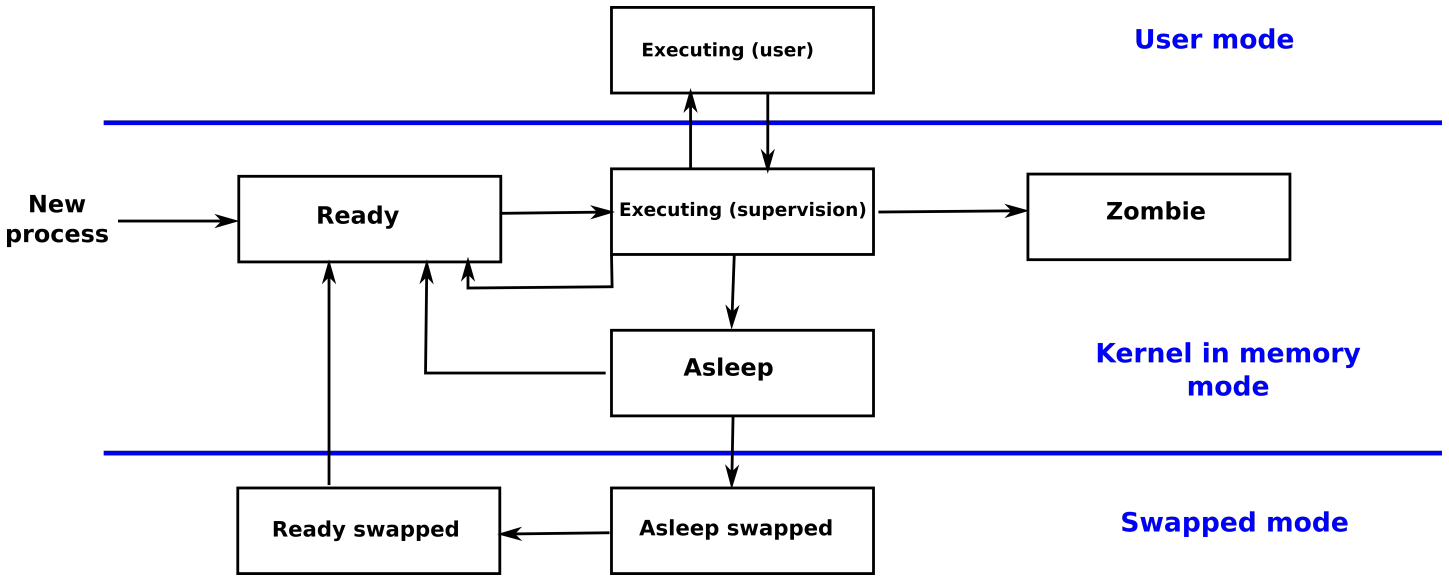
With UNIX each process has an entry in a **table of processes**, with[3]:

- the pid of the process (process identifier)
- the state of the process
- scheduling information
- the state of the memory
- etc.

[3] Basically these are the PCB information...

UNIX processes have *three modes*:

- The **user mode**: classic execution mode for *elected processes*
- The **kernel *in memory* mode**: mode for *elected processes* in *supervision* mode, *ready processes* or *blocked processes*[4]
- The **swapped mode**: processes blocked for a too long time are placed into mass storage memory[5]



[4] Terminated processes go into zombie state waiting for the PCB to be destroyed

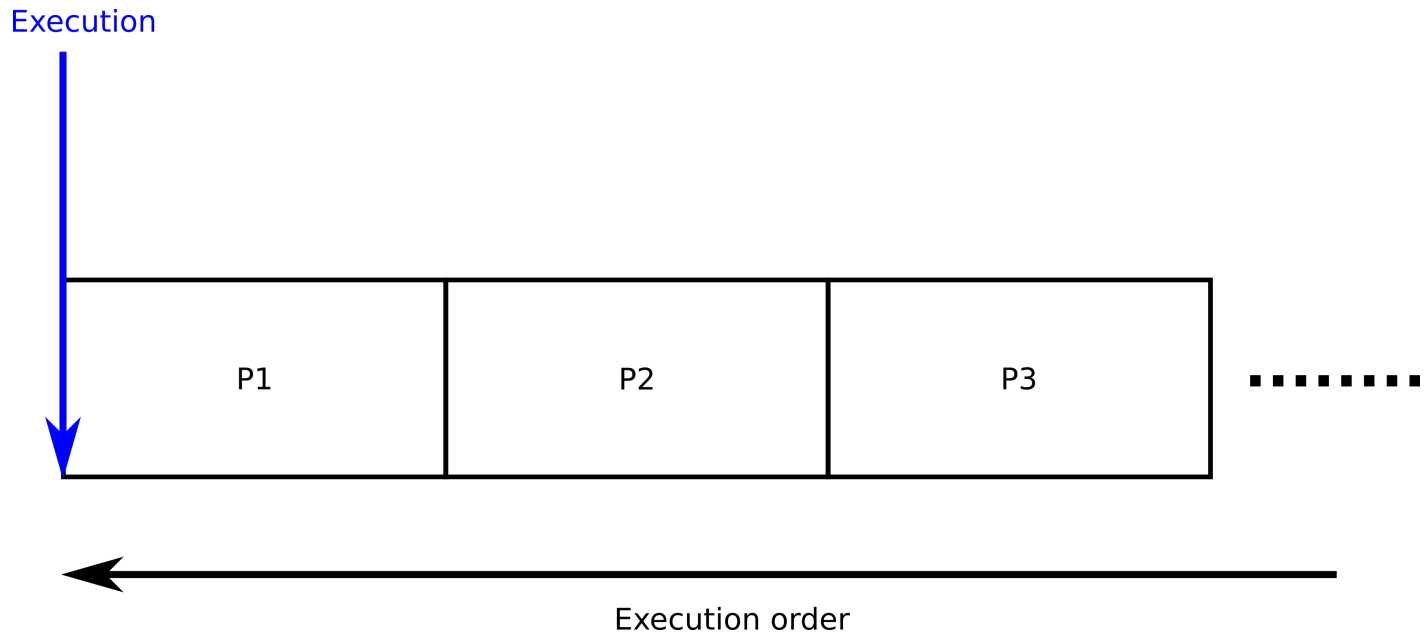
[5] When unblocked they join back the *main memory*, and become *ready*

What does the scheduler?

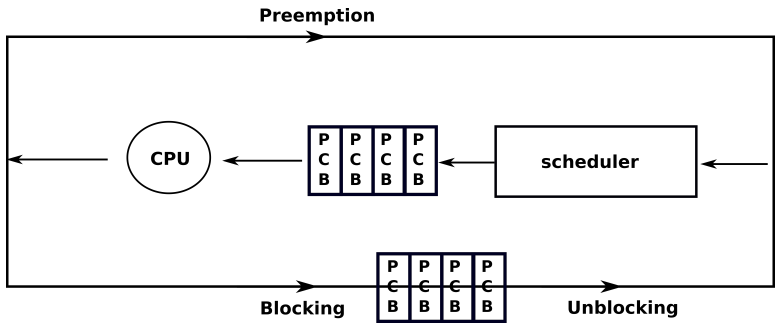
The *scheduler* manages the **priority of execution** of the **processes**

The *scheduler* manages the **priority of execution** by

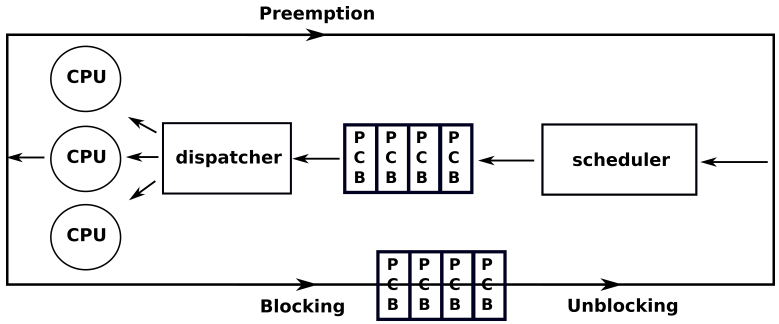
- ...choosing which *process* must be *elected* first
- ...choosing which *processes* must be *elected* after



When dealing with one *processor unit*, mo need of *dispatcher*:



When dealing with several *CPUs*, there is a *dispatcher*:



Preemption

A *preemption* is when the *scheduler* **force a process to leave the CPU**

Policy?

The *scheduler* **prioritize the order of execution** using a *policy*:

- The goal is to **optimize the overall time of execution**
- There are **different policies** for doing that

Among the different possible *policies* there are:

- The **"First in, first executed"** *policy*
- The **"Smallest time of execution first"** *policy*
- The **"Biggest priority first"** *policy*
- The **"Rate monotonic"** *policy*
- The **"Round robin"** *policy*

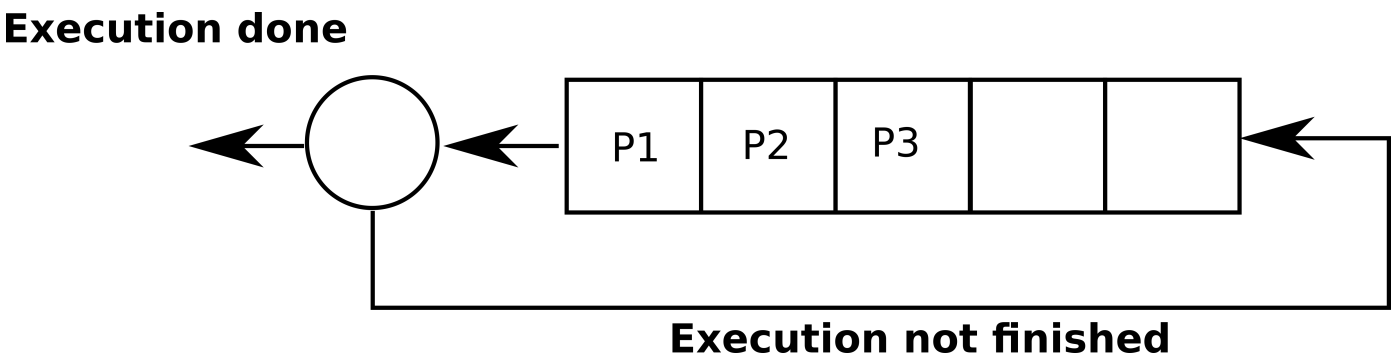
- The **"First in, first executed"** *policy*:
  - *Processes* executed in order of arrival
  - Simple approach
  - BUT: small *processes* are penalized by big *processes*

- The **"Smallest time of execution first"** *policy*:
  - Smallest *processes* are executed first
  - Here it's the minimal average of time of answer!
  - BUT: you need to know how much time last the *processes*

- The **"Biggest priority first"** *policy*:
  - Each process has a priority number: highest priorities are first
  - BUT: processes with low priorities may be executed very late!

- The **"Rate monotonic"** *policy*:
  - Each *process* has a priority number
  - The priority number is proportionate to the frequency of the *process*
  - *Processes* needing to be launched frequently are in top priority
  - Once ready again, a high frequency *process* will be elected in priority

- The **"Round robin"** *policy*:
  - Each *process* is executed in a specific amount of time (10ms to 100ms)
  - If it lasts more: the *process* is preempted until elected again
  - The whole context is stored in the *PCB* to resume after
  - Preempted *processes* are resumed later using a queue:



With Linux:

- Each *process* has a priority number
- There are three different scheduling *policies*:
  - **SCHED_FIFO**: elect processes with the highest priorities
    - ★ for the most important processes
  - **SCHED_RR**: robin round policy between processes
    - ★ for processes with same priority number!
    - ★ for the most important processes
  - **SCHED_OTHER**: elect processes with the highest priorities
    - ★ for all classes!

Note

- Processes attached to SCHED_FIFO and SCHED_RR are more important
- SCHED_OTHER execute the remaining processes